

Test und Zuverlässigkeit

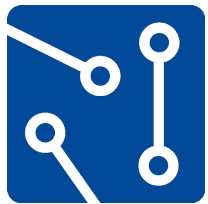
Test and Reliability

Summer Term 2026

Prof. Dr. Ralf Wimmer

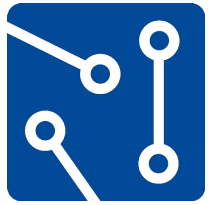
Dr. Tobias Seufert

Chapter 5: Design-for-Testability



Motivation

- The **cost of testing** includes:
 - Test development time
 - Test application cost (tester memory and time)
 - Cost associated with not achieving a high level of fault coverage (Shipping faulty circuits → replacement cost, loss of reputation ...)
- Design-for-testability (DFT) is a general name for design techniques that **reduce** at least one component of **the cost of testing**.
 - For example, DFT can be used to improve fault coverage when it is too low.

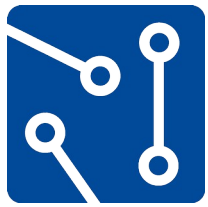


Outline

- Introduction: Design-for-Testability (DFT)
 - vs. Synthesis-for-Testability (SFT)
- Test points
- Sequential circuits
 - Circuit initialization
 - Sequential ATPG
 - Scan

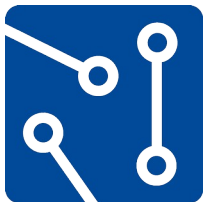
*add additional hardware
to make circuit better
testable*

*synthesize gate netlist
differently (e.g., remove
redundancy) to make it
better testable*



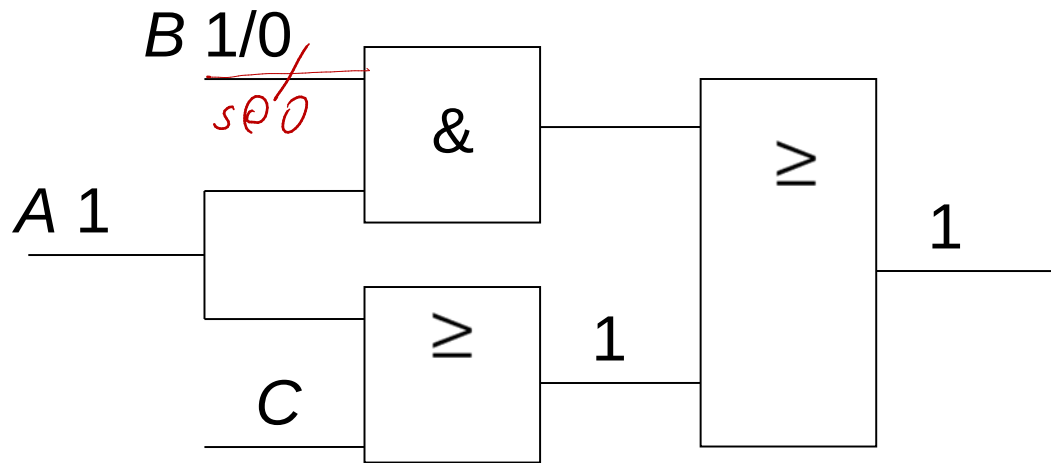
Classes of DFT approaches

- **Design-for-testability** (DFT) involves **adding hardware** to a design to improve testability
- During normal operation, the DFT hardware is **de-activated or bypassed**
- **Synthesis-for-testability** (SFT) involves **re-synthesis** of the design to make it more testable.
- Re-synthesis is not always possible if the design has been optimized for area, speed, or power consumption.



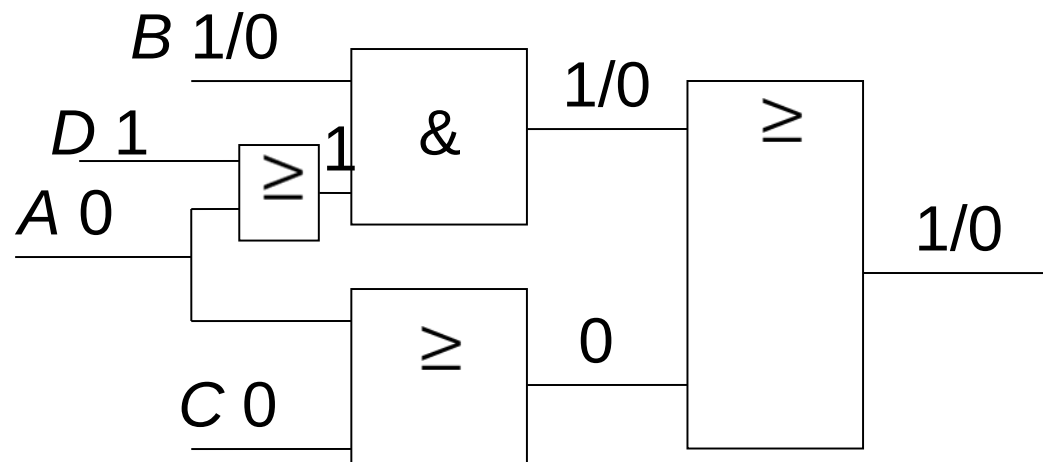
Example (DFT)

B s.a.0 is undetectable

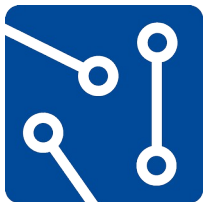


Solution using extra hardware:

D = enable testing mode

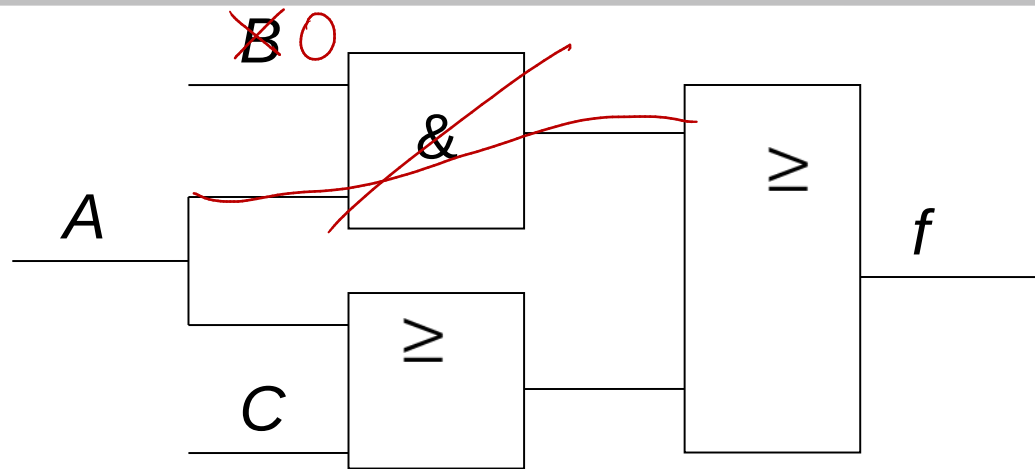


During normal operation, D must be 0.

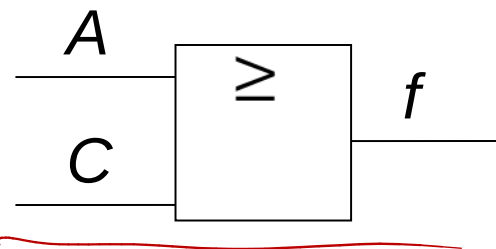


Example (SFT)

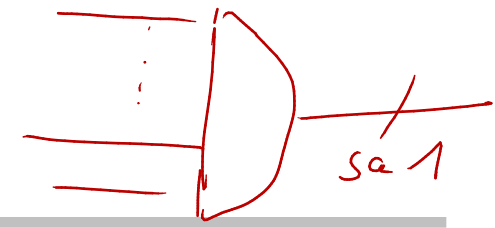
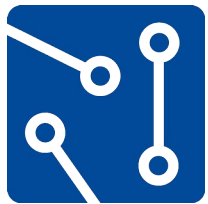
B s.a.0 is
undetectable



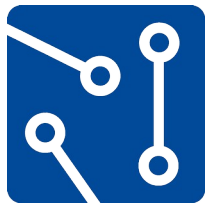
Solution using resynthesis:



Resynthesis might not be desirable because the redundancy was introduced on purpose to make the circuit more robust against transient faults.

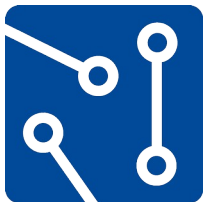


- Circuit structures that are **known to be difficult** to test:
 - Sequential circuits with feedback
 - A gate with a large number of inputs under random patterns
 - Structures with low coverage after ATPG (e.g., due to timeouts)
 - ↪ Need to be targeted to comply with standards
 - Deliberately redundant circuits (→ Reliability)
- Sources of information about testability:
 - Testability measures (inaccurate, → Controllability, Observability)
 - Simulation of random patterns
 - Test coverage
- DFT can target such structures directly



Addressing Specific Problems: Test Points

- **Test point insertion** improves testability
- Two types of test points:
 - **Control points** – primary inputs used to enhance controllability.
 - **Observation points** – primary outputs used to enhance observability.
- Selection of lines for test points:
 - Controllability / Observability measures.
 - Fault simulation of a given test set (via coverage).

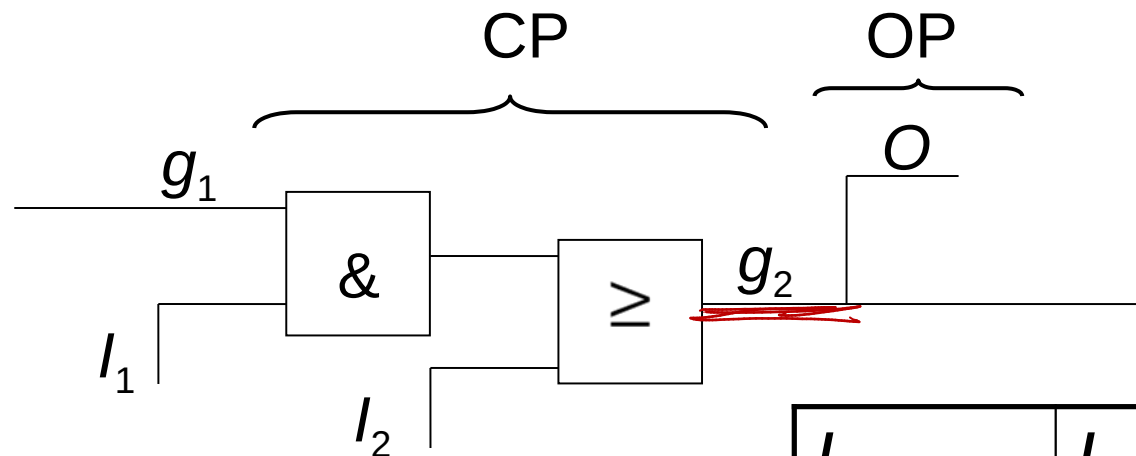


Possible Test Point Structure

g_1 g_2 original circuit

Original circuit

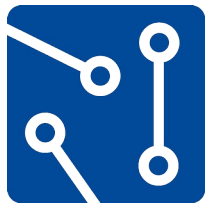
$$g_1 = g_2 = g$$



Options:

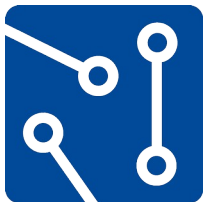
- Observe g_1
- Set g_2 to I_2

I_1	I_2	g_2
0	0	0
x	1	1
1	0	g_1

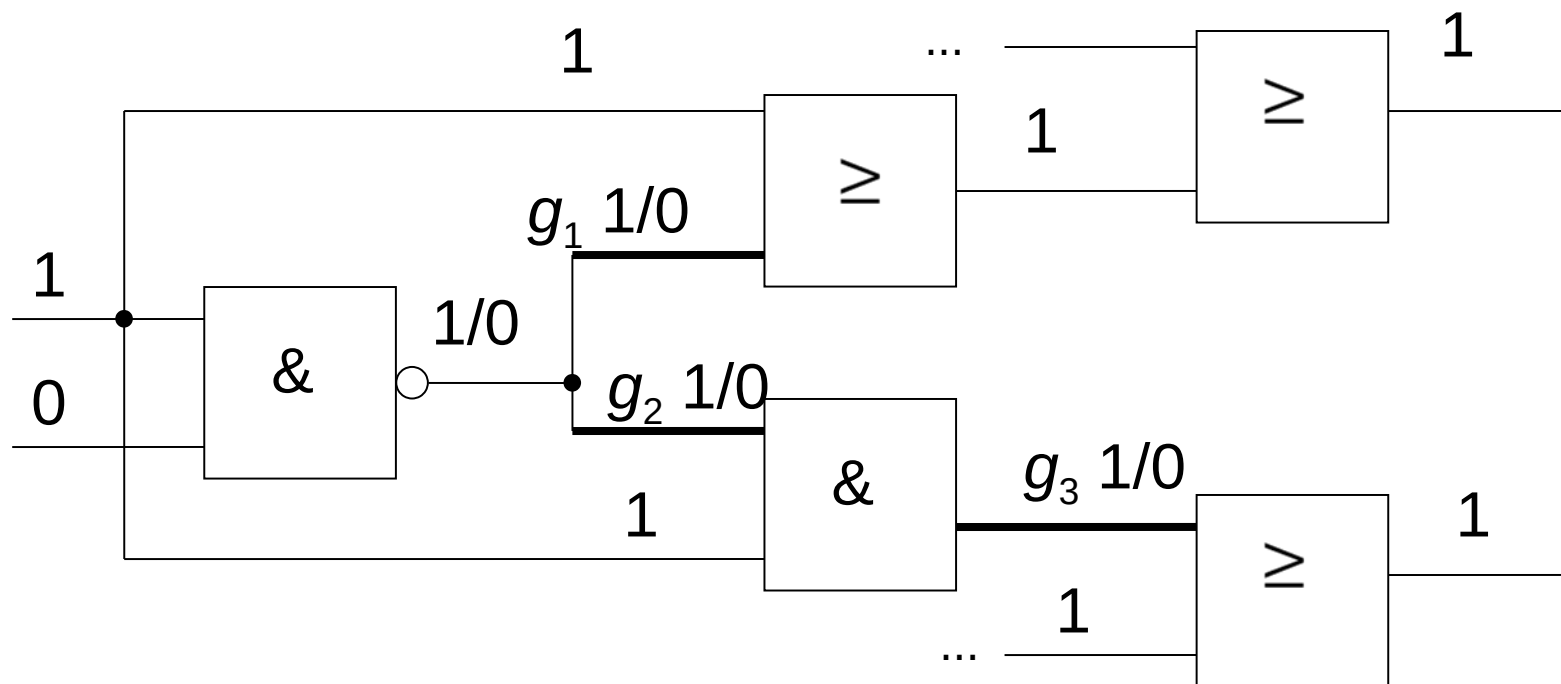


Using Fault Simulation Results

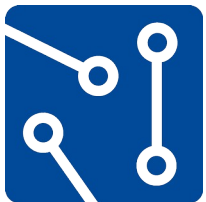
- If a fault f_i can be propagated to a set of internal lines $G_i = \{g_{i1}, g_{i2}, \dots\}$, but not to an output, place an **observation point** on one of the lines in G_i .
- For a set of faults $\{f_1, f_2, \dots, f_k\}$, select a minimum set of lines G such that G contains at least one line out of each G_i .
- If $f_i = l_i / v_i$ cannot be activated, place **control points** on lines that can help set $l_i = v_i$.
- For a set of faults, select the minimum number of control points to activate all of them.



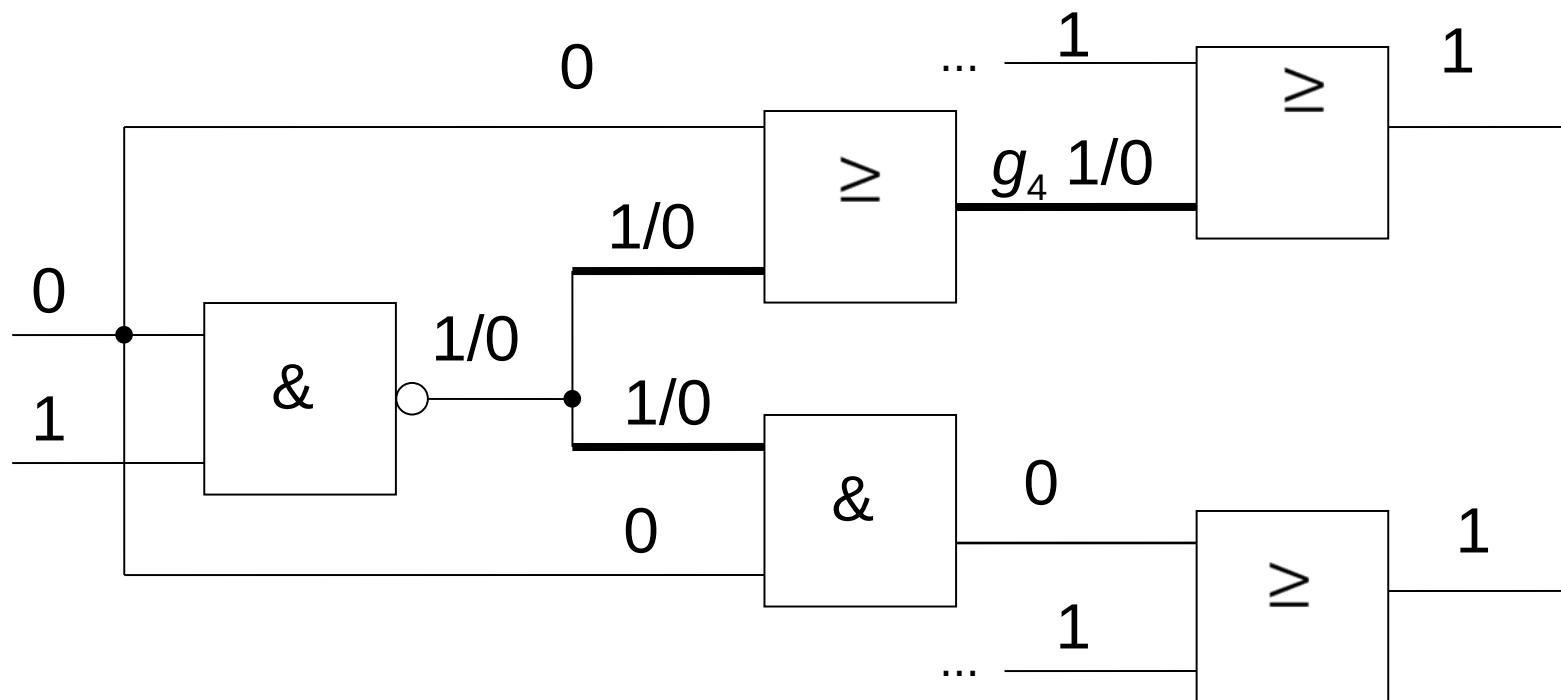
Example Test 1



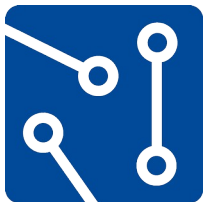
Possible observation points: $G = \{g_1, g_2, g_3\}$



Example Test 2



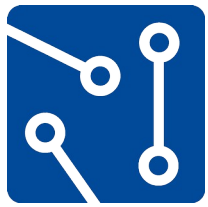
Possible observation points: $G = \{g_1, g_2, g_3, g_4\}$



Considering Several Faults

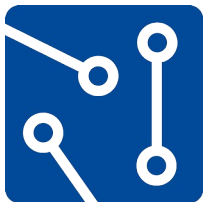
	g_1	g_2	g_3	g_4	g_5	g_6	g_7
f_1	X	X	X				
f_2		X		X			
f_3				X	X	X	X
f_4			X			X	
f_5			X	X			

Set Covering $\rightarrow$ NP-complete



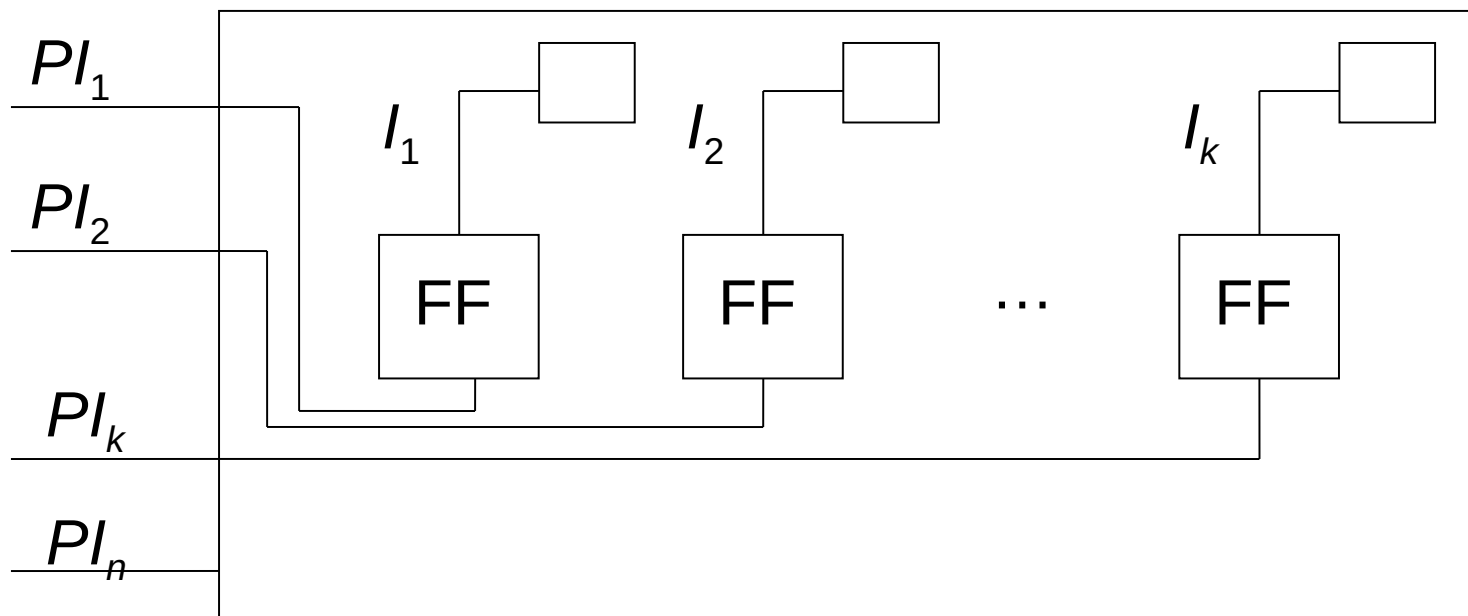
Extra Inputs and Outputs

- Test Points introduce **additional circuit inputs and outputs**
- **Need to be connected** to the outside world to be useful
 - Inputs need to be set **transparent** in normal operations
 - Inputs need to **activate** test points in test mode if required
 - Outputs need to be **observed** as test responses

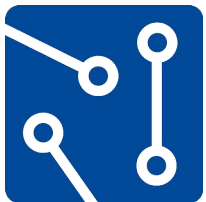


Solution 1 for Extra Inputs

- Add flip-flops to drive the extra inputs.
- Load the flip-flops in parallel from the primary inputs
 - (Almost) no increase in test application time

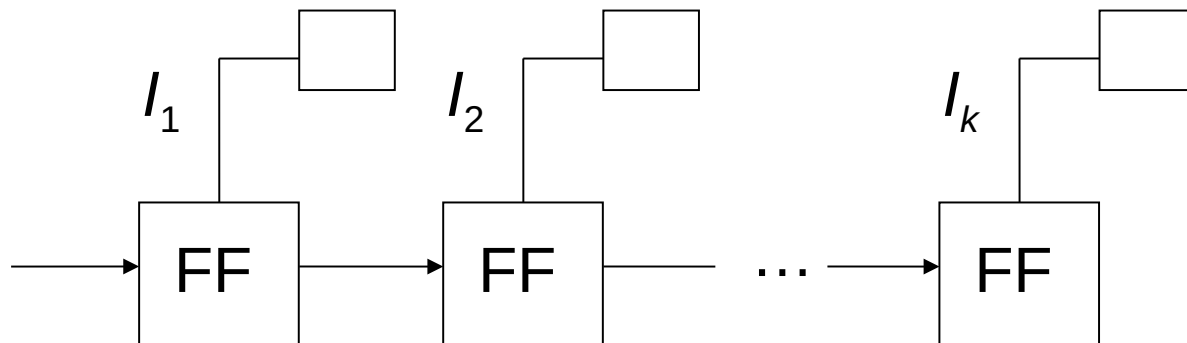


Disadvantage:
Need for IO Pins!

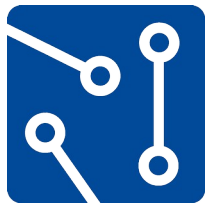


Solution 2 for Extra Inputs

- The number of I/O pins is **limited**.
- Solution 2 for extra inputs:
 - Drive the extra inputs from a **shift register** (scan register).
 - Shift in the required values serially using a single extra input.

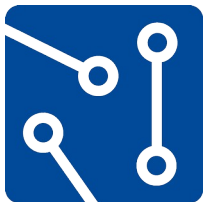


For normal operation, shift in the appropriate values, or set/reset the FFs



Solutions for Extra Outputs

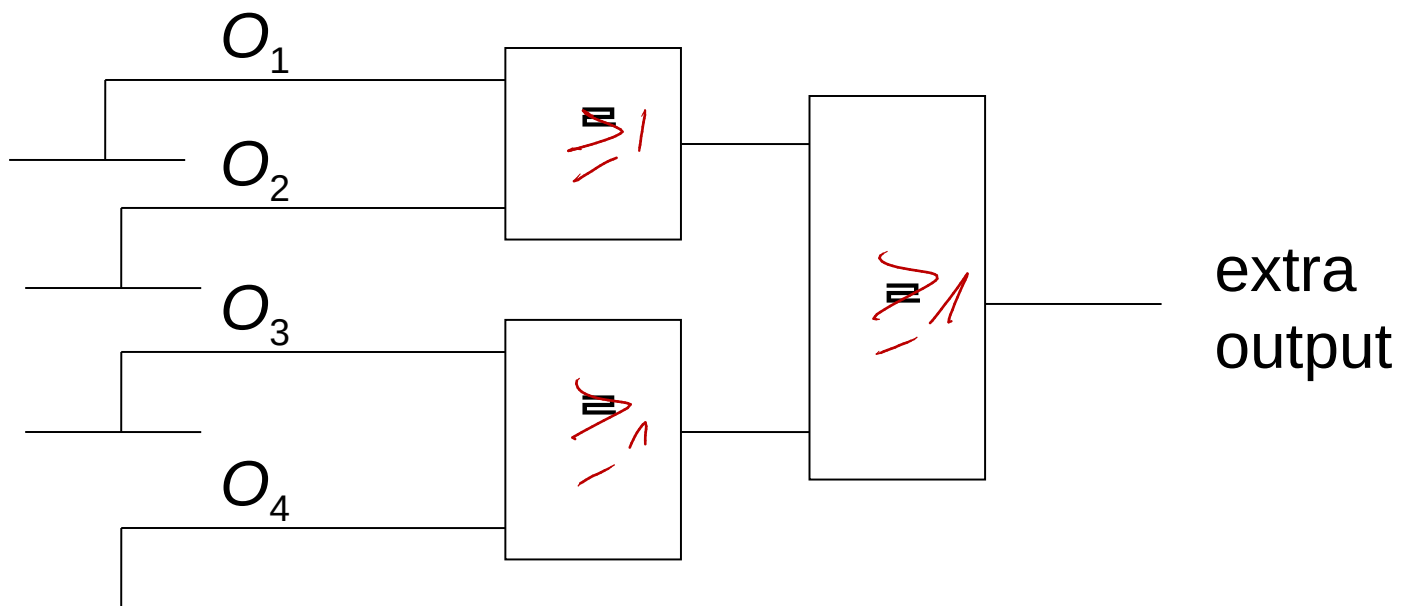
- Solution 1:
 - Connect the extra outputs to a **shift register** and store their values in the flip-flops.
 - Shift out the values serially using a single extra output.
- Solution 2:
 - **Multiplex** the extra outputs with the primary outputs of the circuit.

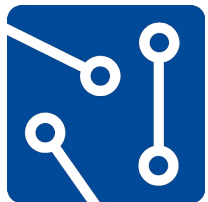


Solution 3 for Extra Outputs

- Connect all the extra outputs to one or more **XOR trees**.
- A fault will be detected if the number of 1's changes its parity.

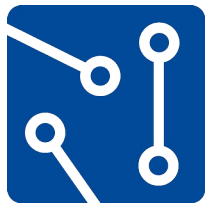
XOR gates





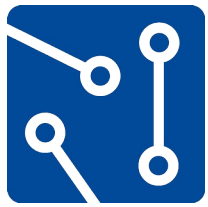
Examples

- If f_1 changes only O_1 under a test t_1 it will be detected on the extra output.
- If f_2 changes only O_2 under a test t_2 it will be detected on the extra output.
- Any fault that changes an **odd number of outputs** under at least one test will be detected.
- If f_3 is detected by one test where it changes O_1 and O_3 it will not be detected. Possible solution: connect O_1 and O_3 to **different XOR trees** and use two extra outputs.



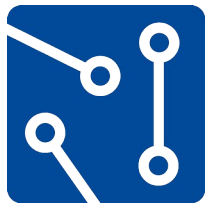
Test Points and BIST

- Use test-points (especially observation points) to **improve fault coverage** achieved by built-in self-test (BIST, see next chapter).
— test pattern generator
- If the hardware TPG cannot achieve complete fault coverage, observation points may be added to detect faults that will otherwise be left undetected.
- Allows relatively simple TPGs to achieve acceptable fault coverage.
- Undetected faults are **identified by simulation** of the TPG sequence. Observation points can be placed based on the results of simulation.



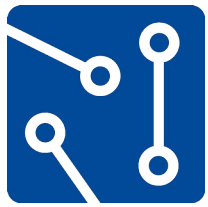
Problems Specific to Sequential Circuits

- Initialization
 - Possible solutions: Partial reset, Scan, Synchronization sequence generation
- Counters
 - Possible solutions: Partitioning, Scan
- Sequential test generation
 - Possible solutions: Scan, Sequential ATPG



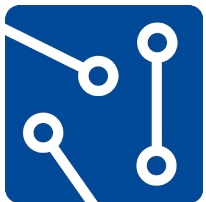
Initialization

- The sequence generated by a sequential circuit depends on the circuit's state
- The **circuit's state** in the beginning of test application is generally **unknown**
- Major problems that rely on known states
 - Test execution depends on state
 - Debugging (bug hunting in layouts) needs traces that end and start in specific states

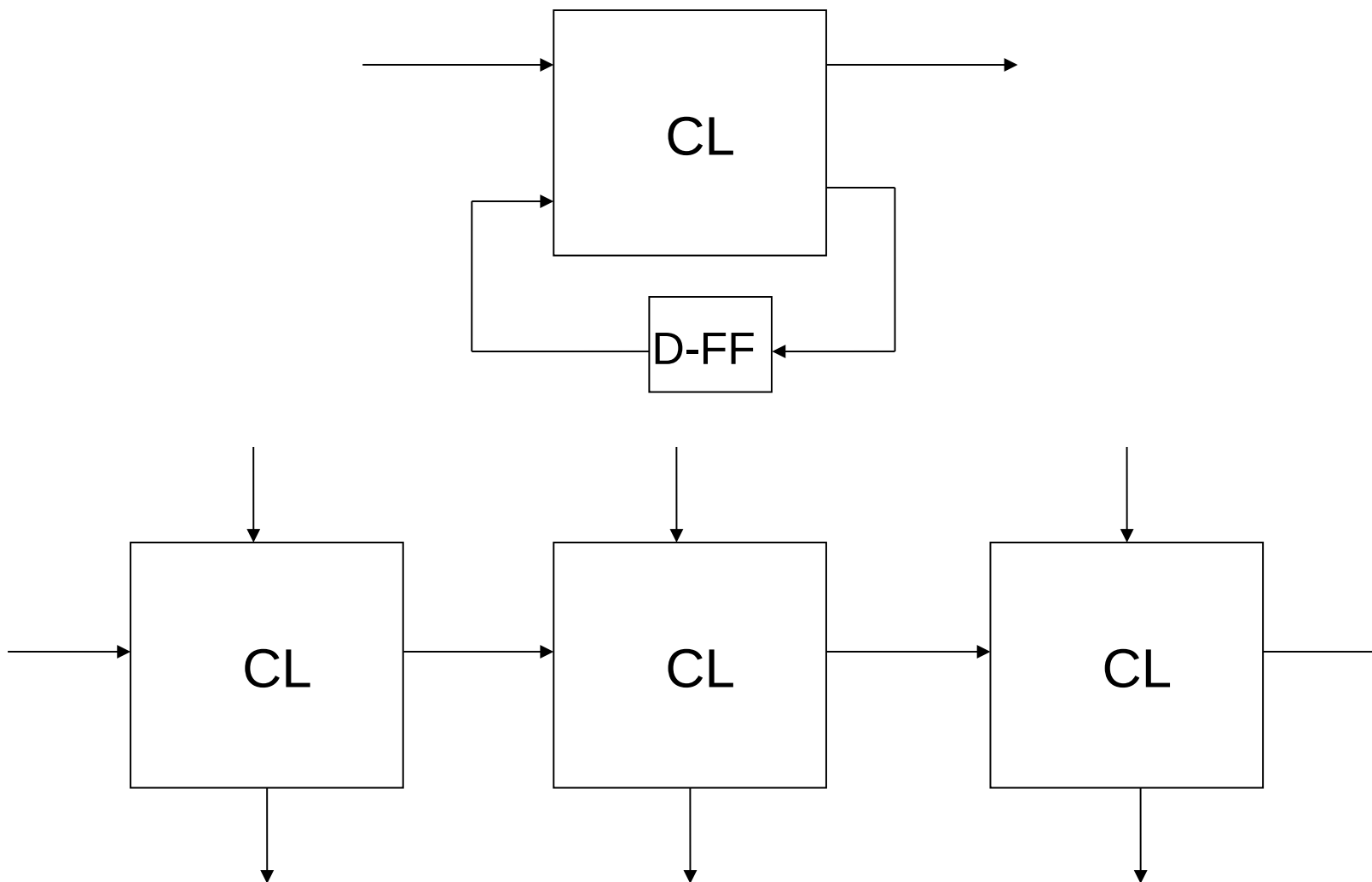


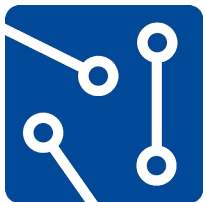
Circuit Initialization

- Initialization – bringing a sequential circuit into a known state.
- Solutions:
 - **Synchronizing sequence** (does not always exist; may be difficult to find even it exists; may be very long [exponential in the number of state bits])
 - **Reset input** (synchronous or asynchronous; requires routing of a set/reset line to all the flip-flops).
 - **Power-up reset** (also requires extra hardware).



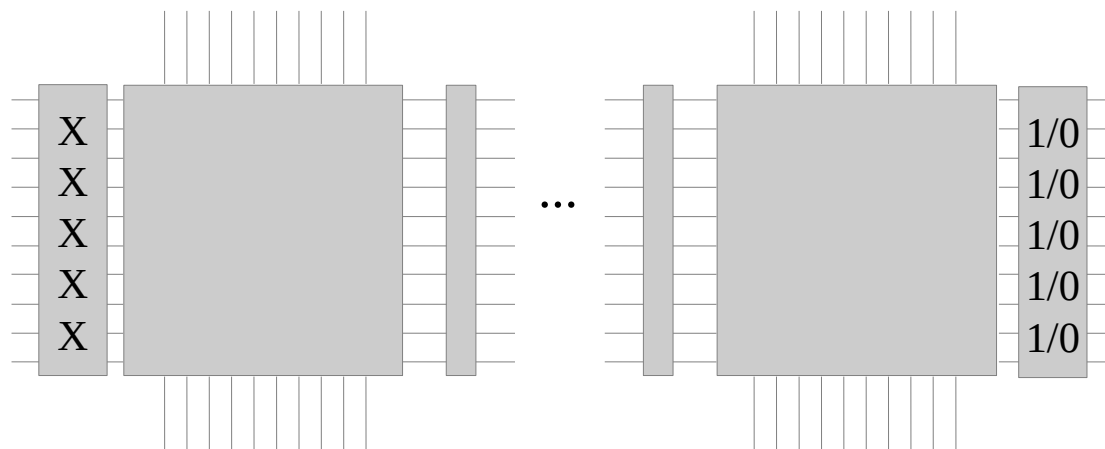
The Iterative Logic Array Model

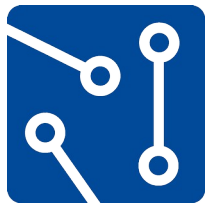




Synchronizing Sequence Generation

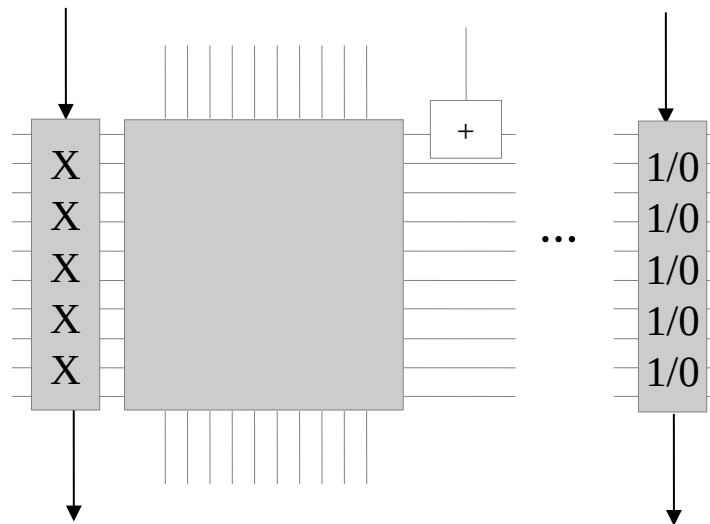
- Bounded Model Checking (BMC) Formulation
 - Initial state: All-X-state
 - Transfer function: Circuit logic
 - Target property: Known state (no X)

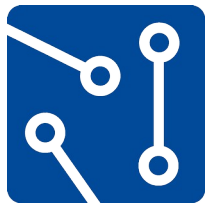




Other Solutions

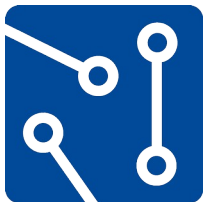
- (Partial) Reset
 - Select a subset of the circuit flip-flops and connect a set/reset input only to them
- Use of Scan-chains to set desired states





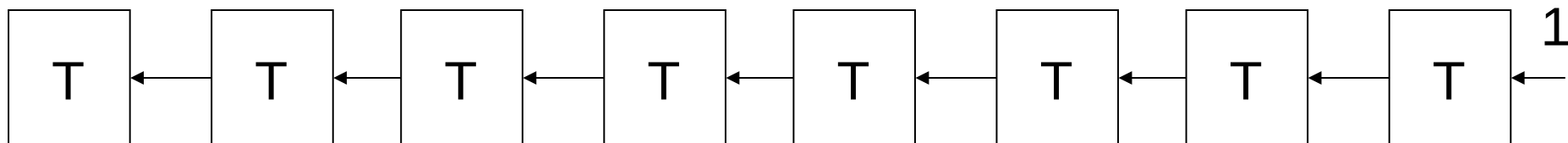
Counters

- To bring a 16 bit counter to the all-1 state can take $2^{16} = 65,536$ clock cycles.
- Current TG procedures cannot generate test sequences of this length.
- Possible Solution: **Partition the counter** into two parts of equal length. Control each part separately.

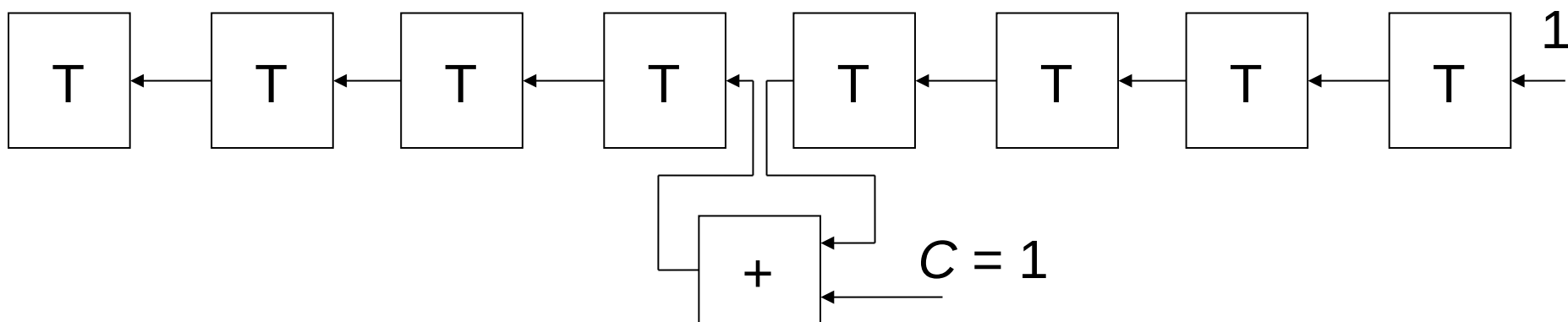


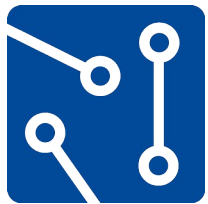
Example of a Counter

Without DFT:



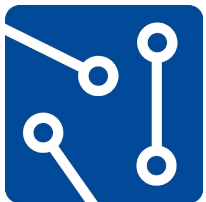
With DFT:





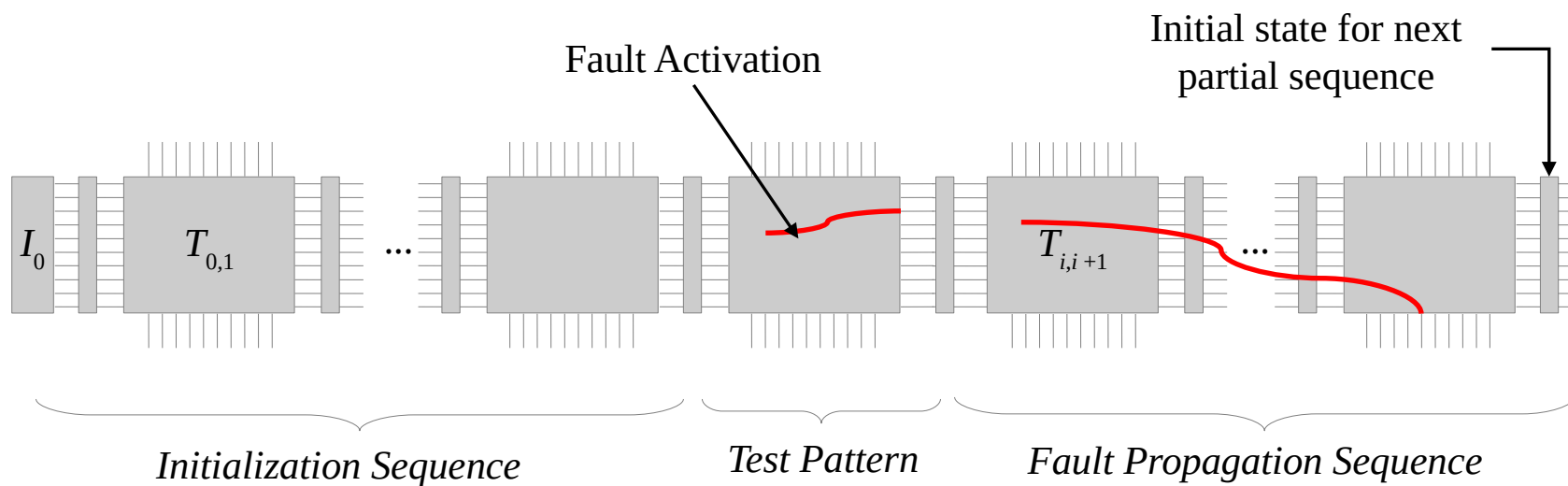
Sequential ATPG

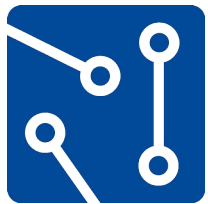
- **Finding a sequence** which distinguishes between a faulty and a fault-free circuit (even if it exists) may require to consider a large number of state transitions
- Practical methods reduce sequential ATPG to combinational ATPG
 - Iterative Logic Array (ILA)



Sequential Test Generation

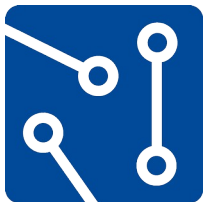
- Set required initial state
- Sensitize the fault
- Propagation of fault effect to observable output



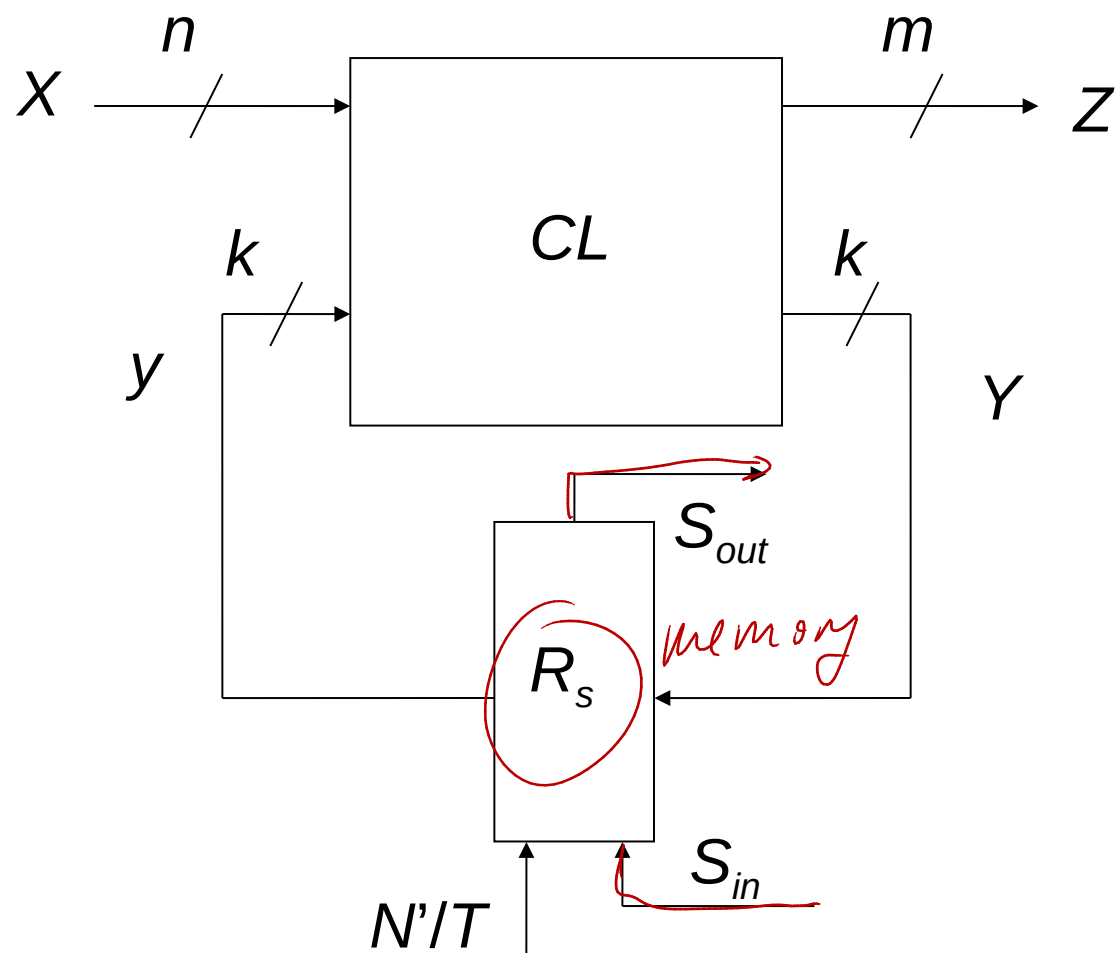


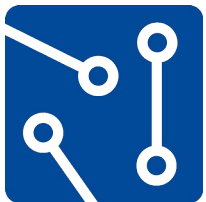
Scan

- The most common DFT technique for sequential circuits.
- Includes:
 - Integrated scan
 - Boundary scan
- Scan can be replaced by parallel load from primary inputs (to primary outputs). However, parallel load has a **significant area overhead** related to routing of multiple lines from the primary inputs (to the primary outputs).

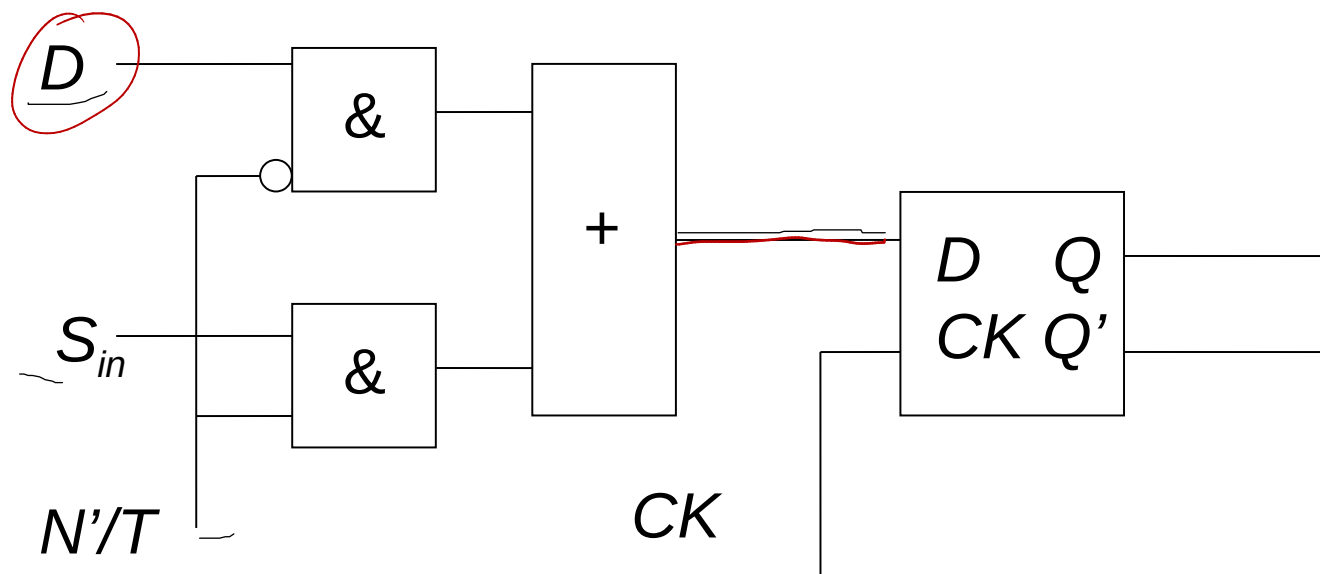


Full Serial Integrated Scan Design



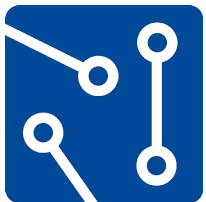


Scan Cell (I)

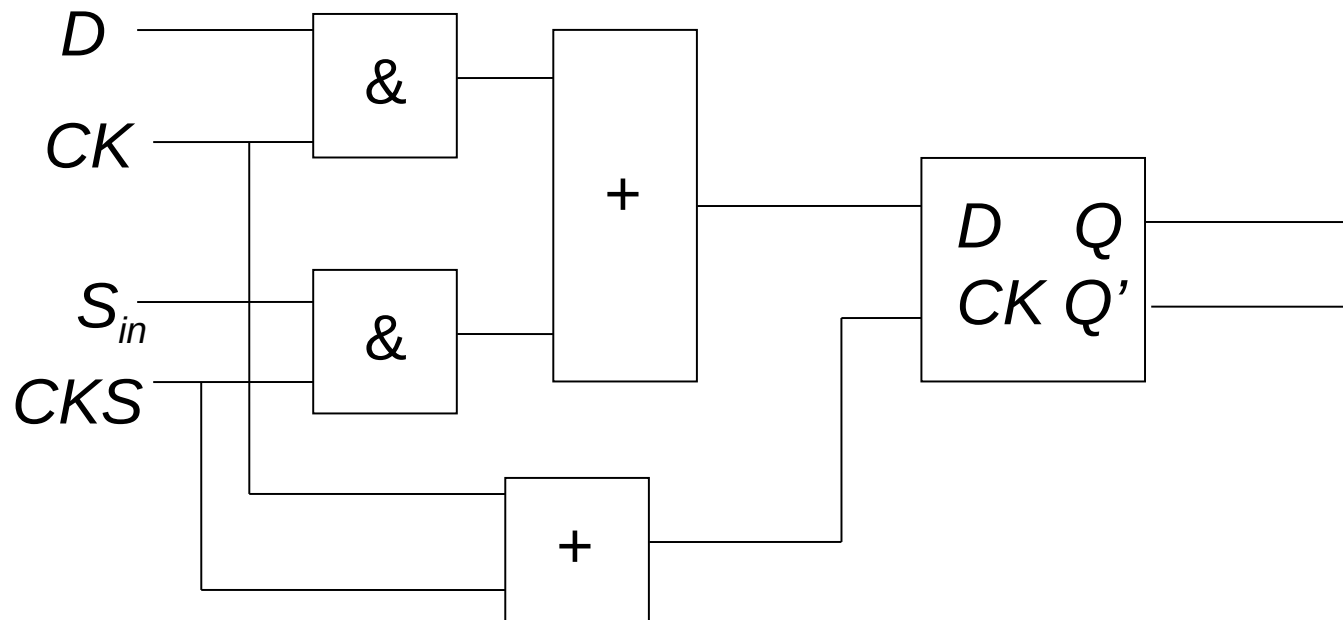


D	Clk	Q	Q'
X	0	No change	
1	1	1	0
0	1	0	1

Same clock for normal operation and test

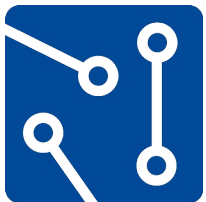


Scan Cell (II)



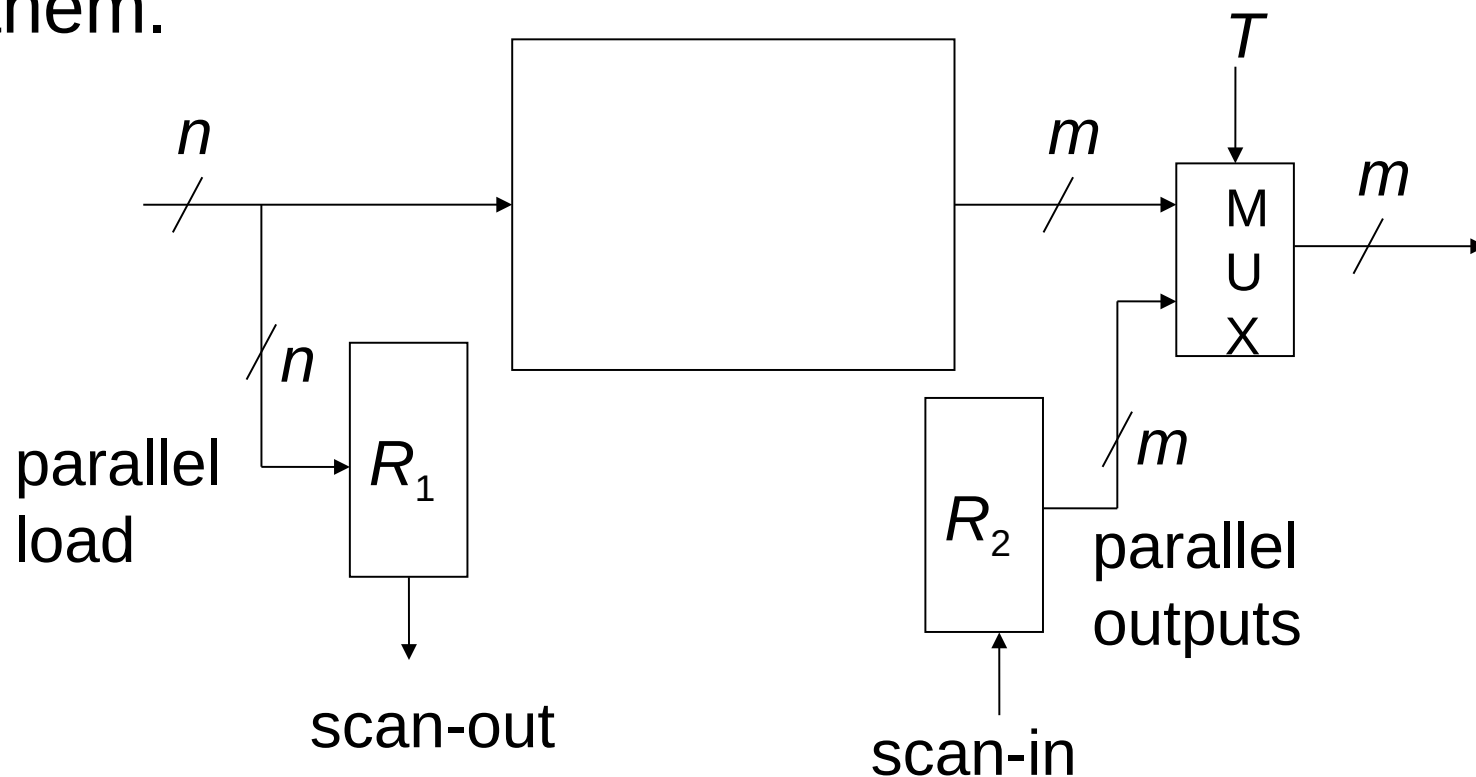
D	Clk	Q	Q'
X	0	No change	
1	1	1	0
0	1	0	1

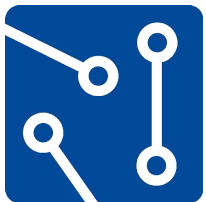
One clock for normal operation and
a separate one for test



Boundary Scan

- Used to isolate different modules in a circuit.
- Can be used to test the modules and the interconnects between them.





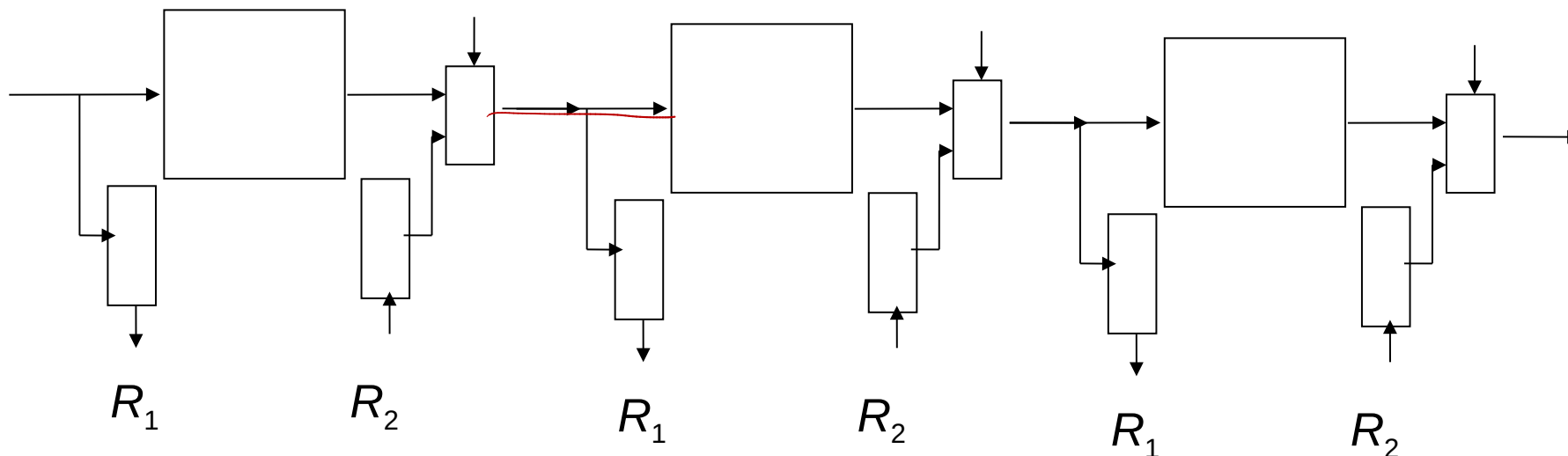
Boundary Scan: Circuit View

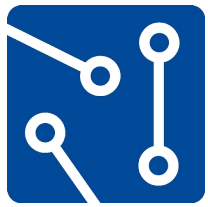
■ Testing **interconnects**:

- Scan in data into R2 of one module.
- Latch the results into R1 of the next module.
- Scan out the data stored in R1.

■ Testing a **module**:

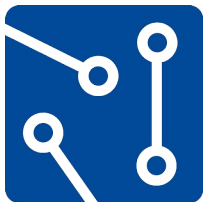
- Scan in data into R2 of the previous module.
- Latch the results into R1 of the next module.
- Scan out the data stored in R1.





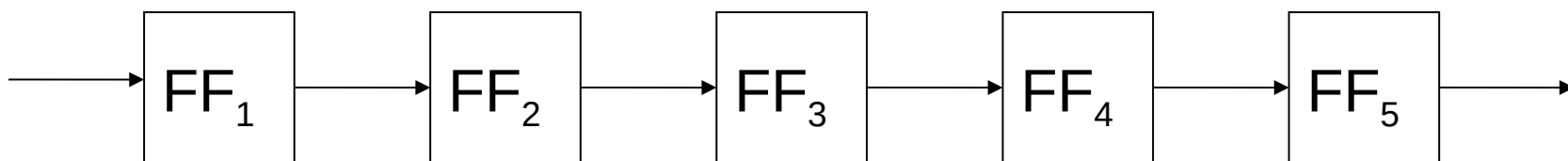
Partial Scan

- Include a **subset** of the circuit flip-flops in the scan chain.
- The subset is usually determined from layout and performance considerations, and it is close to 100%.
- Assuming we have the flexibility of selecting the subset of scan flip-flops, select the subset such that it is as small as possible but allows test generation to be carried out efficiently.
- Partial scan selection methods:
 - Structural.
 - Based on ATPG.



Full Scan vs. Partial Scan

Full scan:



Partial scan:

